AI Predictive Maintenance System (VV for Utilities / AgriPulse)

Full project documentation: pipeline, architecture, and sequential task plan. Graduation Project, AI Engineering Program, Faculty of Engineering, Mansoura University.

1. Project Overview

1.1 Problem

Large centrifugal pumps are the backbone of pressurized irrigation, land reclamation, and water conveyance. When they fail unexpectedly, the result is long agricultural downtime, crop water stress, expensive emergency repairs, and wasted energy and water. Most maintenance today is either reactive (fix after failure) or calendar-based (service on a fixed schedule regardless of condition).

1.2 Solution

The system is an end-to-end, AI-powered Prognostics and Health Management (PHM) platform for irrigation pumps. It learns what normal pump behavior looks like from multi-sensor telemetry, detects deviations early, converts them into an interpretable and calibrated failure risk, explains which sensors drive that risk, and then turns the explanation into an actionable, standards-grounded maintenance work order. A conversational AI Assistant sits on top so technicians can query the whole system in natural language.

1.3 Objectives

- Detect degradation and forecast failure risk with a meaningful early-warning window (target: 6–12 hours before failure).
- Output a graduated, calibrated risk level rather than a binary alarm.
- Explain every alert: which sensors, what pattern, and what physical condition it may indicate.
- Generate maintenance work orders grounded in retrieved documents (ISO 10816-3, O&M manuals, FMEA) with citations and safety (LOTO) steps.
- Provide an operations dashboard and an AI Assistant for monitoring, investigation, and reporting.
- Demonstrate the full loop live through a replay mode and a hardware prototype.

1.4 Core Layers

1. Anomaly detection engine (learns normal behavior, flags deviations).
2. Risk scoring model (calibrated failure probability with graduated levels, plus Remaining Useful Life estimate).
3. Explainability layer (SHAP and per-sensor reconstruction error).
4. Cognitive maintenance layer (RAG-based diagnosis and work-order generation).
5. Operations layer (backend, dashboard, AI Assistant, alerting, IoT edge).

2. Scope, Assumptions, and Known Limitations

In scope (core): data pipeline, anomaly detection, risk scoring, RUL, explainability, evaluation, RAG engine, AI Assistant, backend API, database, dashboard, replay demo, Docker deployment, documentation.

In scope (supporting): IoT edge prototype (ESP32 / Raspberry Pi), voice commands, bilingual (Arabic / English) interface.

Stretch: YOLOv8 computer vision for foliar disease and canopy water stress. It is loosely coupled to the core pump pipeline and should only be attempted once the core is complete.

The following points must be stated honestly in the final report:

- **Dataset:** about 220,320 minute-level records across 52 sensors. If the sensors are anonymous (for example, sensor_00 to sensor_51), physical interpretations are hypotheses based on patterns, not confirmed facts.
- **Few failure events:** real pumps fail rarely, so the dataset likely contains only a handful of failure episodes. Evaluation must be event-level and leave-one-failure-out, and results must be reported with that caveat.
- **Prototype vs. dataset mismatch:** the IoT testbed has about 5 sensor types, while the trained model uses 52. The prototype therefore demonstrates the pipeline (sensing, transport, dashboard, alerting) using replay or a small dedicated model, not the full 52-sensor model.
- **Impact claims** (for example, downtime hours or repair-cost reduction percentages) need a cited source or must be presented as projected estimates.
- **Currency:** the budget must state its currency explicitly.

3. System Architecture

3.1 High-Level View

The platform has six tiers, from data source to user:

1. **Data sources:** historical dataset (training and replay), live IoT telemetry (ESP32 over MQTT), document corpus (ISO standards, manuals, FMEA).
2. **Ingestion:** MQTT broker (Mosquitto) and a replay service that streams historical data as if it were live.
3. **Storage:** PostgreSQL with TimescaleDB for telemetry and application data, Redis for caching and pub/sub, ChromaDB for the vector store, object storage for model artifacts and generated PDFs.
4. **Intelligence services:** inference service (anomaly, risk, RUL, SHAP), RAG service, AI Assistant (agent with tools).
5. **Application layer:** FastAPI backend (REST and WebSocket), authentication and role-based access control.
6. **Presentation:** web dashboard hosted on Vercel, work-order PDFs, alerts.

3.2 Data Flow (Runtime)

1. A telemetry reading arrives (MQTT from the edge device, or from the replay service).
2. Ingestion validates it and writes it to TimescaleDB; it is also pushed to Redis.
3. The inference service maintains a rolling window per pump, computes features, and produces anomaly score, risk probability, risk level, RUL, and top contributing sensors.
4. Results are stored in the predictions table and broadcast over WebSocket to the dashboard.
5. When risk crosses a threshold and persists for N consecutive windows, an alert is created.
6. The alert triggers the RAG service, which builds a query from the top sensors, risk level, and trend, retrieves documents, and generates a structured work order.
7. The technician reviews the alert, the explanation, and the work order in the dashboard, or asks the AI Assistant about it. Feedback (true alarm or false alarm) is stored for later retraining and threshold tuning.

3.3 Components and Responsibilities

- **Data & Feature Service:** cleaning, labeling, windowing, feature computation (shared code for training and serving to avoid training/serving skew).

- **Inference Service:** loads versioned models, exposes a single predict function, computes SHAP values.
- **RAG Service:** document ingestion, chunking, embeddings, hybrid retrieval, reranking, LLM generation with schema-constrained output.
- **AI Assistant:** an LLM agent with function-calling tools that call the backend, never answering numeric questions from its own memory.
- **Backend API:** the single gateway for the frontend and the assistant; handles auth, validation, and orchestration.
- **Frontend:** SCADA-style dashboard and assistant chat.
- **Edge:** ESP32 reads sensors and publishes JSON over MQTT; Raspberry Pi acts as a gateway and optional local inference node.

3.4 Technology Stack

- **ML:** Python 3.11, PyTorch, scikit-learn, XGBoost, LightGBM, SHAP, MLflow.
- **RAG and LLM:** LangChain or LlamaIndex, ChromaDB (or FAISS), a multilingual embedding model, an LLM via Gemini, OpenAI, or Ollama, Whisper for speech.
- **Backend:** FastAPI, WebSockets, Pydantic schemas, JWT authentication.
- **Data:** PostgreSQL with TimescaleDB, Redis.
- **Frontend:** React (Next.js) on Vercel, Plotly.js or Chart.js, i18n for Arabic and English with RTL support.
- **Infrastructure:** Docker and Docker Compose, GitHub Actions, a cloud VM (AWS, Azure, or GCP) or a container platform for the backend, Mosquitto MQTT.

3.5 Deployment Topology

- **Vercel:** frontend only.
- **Container host (VM or container platform):** backend, inference service, RAG service, Redis, MQTT broker.
- **Managed database:** TimescaleDB or PostgreSQL (for example, Timescale Cloud, Neon, or Supabase).
- The backend is not deployed on Vercel because it needs long-lived WebSocket connections and loads heavy ML libraries.

3.6 Security

- JWT authentication, with tokens delivered in httpOnly cookies.
- Role-based access control with three roles: Admin, Technician, Agronomist.
- HTTPS/TLS everywhere; MQTT with credentials and TLS.

- Secrets stored in environment variables or a secrets manager, never in the repository.
- Input validation on all endpoints; rate limiting on assistant and RAG endpoints.
- Audit log for alerts, work orders, and feedback.

3.7 Interface Contracts (Fixed Early)

These contracts are agreed before parallel development begins:

1. **Prediction JSON:** `{pump_id, timestamp, anomaly_score, risk_prob, level, rul_hours, top_sensors[{sensor, group, shap_value}], model_version}`.
2. **Work-order JSON:** `{alert_id, probable_causes[], recommended_actions[], urgency, loto_steps[], parts[], estimated_downtime, citations[]}`.
3. **OpenAPI specification** for all REST and WebSocket endpoints.
4. **Database schema** (Section 8).
5. **Assistant tool schemas** (Section 6).

4. Machine Learning Pipeline

4.1 Data Understanding and Cleaning

- Load the dataset; verify timestamps are regular and sorted; detect duplicates and gaps.
- Profile each of the 52 sensors: missing rate, variance, range, outliers. Drop sensors that are empty or constant; document each decision.
- If the data includes a machine status column (for example, NORMAL, RECOVERING, BROKEN), use it for labeling. Recovery periods are excluded from both training and evaluation because they are neither normal nor pre-failure.
- Impute short gaps (forward fill or interpolation with a maximum gap length); flag long gaps rather than inventing data.
- Scaling: fit scalers on the training portion only to prevent leakage.

4.2 Labeling

- **Pre-failure window label:** for every failure event, mark the 6 to 12 hours before it as the positive class; all other normal-operation time is negative. The window length is a hyperparameter evaluated at several values (for example, 6, 8, and 12 hours).
- **RUL target:** for samples in the run-up to failure, the time remaining until the failure, capped at a maximum horizon.
- Anomaly detectors are trained on normal data only and need no failure labels.

4.3 Splitting and Validation

- **Never use random splits** on time-series windows; adjacent samples are almost identical and would leak.
- Use **leave-one-failure-out** (or blocked time-based) cross-validation: each fold holds out one complete failure episode and the normal period around it.
- Add a gap (purge) between training and validation windows equal to the longest rolling window.

4.4 Feature Engineering

- **Rolling statistics** over multiple windows (for example, 15 min, 1 h, 6 h): mean, standard deviation, min, max, range, skewness, and slope.
- **Exponentially weighted** mean and variance for fast-reacting trends.
- **Frequency-domain features** (FFT, spectral energy, dominant frequency) for sensors that behave like vibration or current signals.
- **Cross-sensor features:** correlations within sensor groups, ratios, and PCA reconstruction residuals.
- **Sensor grouping:** cluster sensors by correlation into groups (for example, pressure-like, thermal-like, electrical-like). Groups are used for explainability and are labeled neutrally if sensor semantics are unknown.
- The same feature code is used in training and serving.

4.5 Layer 1: Anomaly Detection

- **Primary model:** LSTM-Autoencoder trained on normal windows. The anomaly score is the reconstruction error; per-sensor error gives a first explanation.
- **Baselines for comparison:** Isolation Forest, PCA reconstruction error, Mahalanobis distance.
- The threshold is chosen from the distribution of scores on normal validation data (for example, a high percentile), then smoothed with an exponential moving average.

4.6 Layer 2: Risk Scoring

- **Model:** XGBoost and LightGBM on engineered features plus the anomaly score. A Bidirectional LSTM is trained as a deep-learning alternative for sequence input.
- **Imbalance handling:** class weights or focal loss; no synthetic oversampling that would distort time structure.
- **Calibration:** isotonic regression or Platt scaling so that a score of 0.7 means roughly a 70% observed frequency.

- **Graduated levels:** Green (normal), Yellow (watch), Orange (degrading), Red (critical), defined by calibrated probability thresholds.
- **Persistence logic and hysteresis:** a level changes only after N consecutive windows above or below the threshold, to reduce alert fatigue.
- **RUL estimation:** gradient boosting or LSTM regression, evaluated with MAE and a scoring function that penalizes late predictions more than early ones.

4.7 Layer 3: Explainability

- **TreeSHAP** on the risk model gives per-prediction contributions of each feature, aggregated up to sensor and sensor-group level.
- **Reconstruction error per sensor** from the autoencoder shows which signals deviate from normal.
- **Output for the user:** top 5 contributing sensors, direction of deviation, how long it has been drifting, and a plain-language summary.
- **Interpretation rule:** map patterns to physical hypotheses (for example, a drifting thermal group with rising load may suggest bearing or cooling degradation) and label them as hypotheses, not diagnoses. Hypotheses are validated against the FMEA knowledge base in the RAG layer.
- **Global explainability:** SHAP summary plots and group importance for the final report.

4.8 Evaluation

Window-level metrics (precision, recall, F1, PR-AUC) are reported but are not the headline. The headline metrics are event-level:

- **Detection rate:** fraction of failures preceded by an alert at or above Orange.
- **Lead time:** how many hours before failure the first qualifying alert fired (mean, median, minimum).
- **False alarms:** alerts per week during normal operation.
- **Calibration:** reliability diagram and Brier score.
- **RUL error:** MAE and the asymmetric score.
- **Ablations:** anomaly-only vs. supervised-only vs. combined; with and without persistence logic; with and without frequency features.
- All results are reported per held-out failure, with the small number of events stated clearly.

4.9 Model Management

- MLflow tracks experiments, parameters, metrics, and artifacts.

- Each model is versioned and stored with its scaler, feature list, thresholds, and a model card (intended use, data, metrics, limitations).
- A single `predict()` interface wraps the full chain: windowing, features, anomaly score, risk, RUL, SHAP.

5. Cognitive Maintenance: RAG Engine

5.1 Knowledge Base

- **Sources:** ISO 10816-3 vibration severity guidance, manufacturer O&M manuals for centrifugal pumps, and an FMEA matrix (failure mode, cause, effect, detection, recommended action, severity) authored by the team from standard references.
- **Processing:** convert to clean text, split by section and table row (an FMEA row is one chunk), attach metadata (source, section, page, document type), and embed with a multilingual embedding model so both English and Arabic queries work.

5.2 Retrieval

- **Hybrid retrieval:** BM25 keyword search plus dense vector search, merged with reciprocal rank fusion.
- **Reranking:** a cross-encoder or LLM reranker on the top candidates.
- **Query construction:** automatically built from the alert (top sensors, groups, deviation direction, risk level, trend), then optionally expanded into several sub-queries (failure modes, severity thresholds, actions).

5.3 Generation

- The LLM receives only retrieved passages plus the structured alert context.
- Output is schema-constrained JSON (the work-order contract) and is validated; invalid output is retried.
- **Citations are mandatory:** every cause, action, and safety step points to a source chunk.
- **Safety rules:** Lockout/Tagout steps are copied from retrieved sources, never invented; if retrieval confidence is low, the work order says so and recommends manual inspection.
- **Human in the loop:** a work order is a draft until a technician approves it.
- The approved work order is rendered to a PDF.

5.4 RAG Evaluation

- A small gold set of alert scenarios with expected causes and actions written by the team.
- Metrics: retrieval hit rate at k, faithfulness (is every claim supported by a retrieved chunk), citation correctness, and a manual quality rubric.

6. AI Assistant

6.1 Design

An LLM agent with function calling. It answers only through tools, so every number, status, and recommendation comes from the system.

6.2 Tools

- `get_risk_status(pump_id, time_range)`: current level, probability, trend, RUL.
- `explain_alert(alert_id)`: SHAP top sensors, groups, and summary.
- `query_telemetry(pump_id, sensor, window)`: returns data and a chart specification.
- `search_knowledge_base(query)`: RAG retrieval with citations.
- `generate_work_order(alert_id)`: triggers the RAG work-order pipeline.
- `list_alerts(filters)` and `compare_periods(pump_id, period_a, period_b)`.
- `log_feedback(alert_id, label, comment)`: records true or false alarm.
- `what_if(pump_id, sensor_changes)` (optional): re-scores the risk with modified inputs.

6.3 Behavior and Guardrails

- System prompt defines the role, answer style, language handling (Arabic or English, following the user), and refusal rules.
- Numeric answers must come from a tool result; if a tool fails or returns nothing, the assistant says so instead of guessing.
- Maintenance and safety content must carry citations.
- Actions with real-world consequences require explicit user confirmation.
- Tools enforce the caller's role (a role can only see and do what it is permitted to).
- Conversation memory is kept per session; tool calls are logged for audit.
- Optional voice interface: speech-to-text with Whisper or the Web Speech API, text-to-speech for replies.

7. Backend and API

7.1 Principles

A FastAPI service with Pydantic-validated schemas, asynchronous handlers, structured logging, and a published OpenAPI document that the frontend and assistant consume.

7.2 Endpoints (Representative)

- **Auth:** `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`.
- **Pumps:** `GET /pumps`, `GET /pumps/{id}`, `GET /pumps/{id}/telemetry`, `GET /pumps/{id}/predictions`.
- **Alerts:** `GET /alerts`, `GET /alerts/{id}`, `POST /alerts/{id}/ack`, `POST /alerts/{id}/feedback`.
- **Explanations:** `GET /alerts/{id}/explanation`.
- **Work orders:** `POST /alerts/{id}/work-order`, `GET /work-orders/{id}`, `PATCH /work-orders/{id}` (approve or edit), `GET /work-orders/{id}/pdf`.
- **Assistant:** `POST /assistant/chat` (streaming), `POST /assistant/voice`.
- **Replay:** `POST /replay/start`, `POST /replay/pause`, `POST /replay/speed`.
- **Admin:** `GET /models`, `POST /models/{version}/activate`, `GET /users`, `POST /users`.
- **Realtime:** `WS /stream/{pump_id}` pushes telemetry and prediction updates.
- **Ingest:** MQTT topic `pumps/{pump_id}/telemetry` consumed by an ingestion worker.

8. Database Design

- **pumps:** id, name, location, metadata, status.
- **telemetry (hypertable):** pump_id, ts, sensor values (wide columns or a JSONB payload), source (live, replay).
- **predictions (hypertable):** pump_id, ts, anomaly_score, risk_prob, level, rul_hours, model_version.
- **explanations:** prediction or alert id, top_sensors JSON, group contributions, summary text.
- **alerts:** id, pump_id, level, opened_at, status (open, acknowledged, resolved), assignee, feedback label.
- **work_orders:** id, alert_id, content JSON, status (draft, approved, completed), approved_by, pdf path, timestamps.
- **kb_documents:** id, source, type, version, ingestion date (vectors live in ChromaDB).

- **users and roles:** id, email, password hash, role.
- **audit_log:** actor, action, object, timestamp.
- **model_registry:** version, metrics, artifact path, active flag.
- Timescale features: hypertable partitioning, compression for old telemetry, and continuous aggregates (hourly and daily) for dashboard charts. Redis caches the latest state per pump.

9. Frontend (Vercel)

- **Login** with role-based navigation.
- **Fleet overview:** all pumps with risk level, trend sparkline, and open alerts.
- **Pump detail:** live risk gauge, telemetry charts with selectable sensors, anomaly score timeline, RUL, SHAP bar chart and sensor-group view.
- **Alerts center:** filter, acknowledge, give feedback, open explanation.
- **Work orders:** view, edit, approve, download PDF.
- **AI Assistant page:** chat with streaming replies, inline charts, citations, voice input.
- **Replay control panel:** start, pause, and speed controls to demonstrate early detection.
- **Admin:** users, models, knowledge-base documents.
- **Quality:** responsive layout, Arabic and English with RTL, light and dark themes, loading and error states.

10. IoT Edge Prototype

- **Hardware:** ESP32 with an accelerometer (ADXL345 or MPU-6050), a non-invasive current clamp, a pressure transducer, a temperature sensor, and a flow meter on a small closed-loop centrifugal pump test bench; Raspberry Pi 4 as gateway.
- **Firmware:** sampling, basic filtering, JSON packaging, MQTT publishing with reconnect and timestamping.
- **Edge logic:** the Pi can run a lightweight anomaly model trained on bench data for a self-contained demo.
- **Fault injection for the demo:** controlled, safe disturbances such as partially closing a valve, restricting inlet flow, or adding a small imbalance, to show the dashboard reacting.
- The prototype is positioned as a proof of the sensing-to-dashboard chain, separate from the 52-sensor model evaluation.

11. DevOps, MLOps, and Quality

- Git and GitHub with branch protection and pull-request review.
- Docker images for each service; Docker Compose for local and staging environments.
- GitHub Actions: lint, unit tests, build, and deploy.
- Tests: unit tests for feature code and API, an integration test for the full alert-to-work-order path, and load tests for the WebSocket stream.
- Monitoring: structured logs, health endpoints, data-drift checks on live inputs, and a model-performance dashboard.
- Retraining procedure documented: how feedback and new data are folded in, and how a new model version is promoted.

12. Optional Module: Computer Vision (Stretch)

A YOLOv8 model detects foliar disease and canopy water stress from field images, exposed through an inference endpoint and a dashboard page. It uses a public plant-disease dataset and is evaluated independently (mAP, precision, recall). It is attempted only after the core system is complete.

13. Sequential Task Plan

Tasks are ordered from the first step of the project to the last. Each task lists its workstream (WS) and its deliverable. The workstreams are: WS1 Data & Features, WS2 Models, WS3 Explainability & Evaluation, WS4 RAG Engine, WS5 AI Assistant, WS6 Backend & Auth, WS7 Database, DevOps & IoT, WS8 Frontend.

Phase A: Foundation

- **T01. Define scope, roles, and success metrics (all).** Deliverable: a one-page scope and metrics agreement.
- **T02. Set up the repository, branching rules, and project board (WS7).** Deliverable: monorepo structure, README, issue templates.
- **T03. Set up the development environment (WS7).** Deliverable: Docker dev environment and pinned dependencies.
- **T04. Write the interface contracts (all).** Deliverable: prediction JSON, work-order JSON, and tool schemas reviewed and frozen.
- **T05. Draft the OpenAPI specification (WS6).** Deliverable: openapi.yaml agreed with frontend and assistant owners.
- **T06. Design the database schema (WS7).** Deliverable: ERD and migration scripts.

- **T07. Create the frontend skeleton with mock data (WS8).** Deliverable: Next.js app on Vercel showing mocked fleet and pump pages.

Phase B: Data

- **T08. Ingest and profile the dataset (WS1).** Deliverable: profiling notebook and report (missingness, ranges, anomalies).
- **T09. Clean the data and decide sensor handling (WS1).** Deliverable: documented sensor keep or drop list and imputation rules.
- **T10. Build the labeling logic (WS1).** Deliverable: pre-failure labels and RUL targets with configurable window length.
- **T11. Define the validation scheme (WS1, WS3).** Deliverable: leave-one-failure-out folds with purge gaps.
- **T12. Implement the feature engineering module (WS1).** Deliverable: shared, tested feature library (rolling, EWM, FFT, cross-sensor, groups).
- **T13. Produce and version the feature dataset (WS1).** Deliverable: versioned feature tables and a data dictionary.

Phase C: Models

- **T14. Train baseline anomaly detectors (WS2).** Deliverable: Isolation Forest, PCA, and Mahalanobis results.
- **T15. Train the LSTM-Autoencoder (WS2).** Deliverable: trained model, threshold selection, per-sensor reconstruction error.
- **T16. Train supervised risk models (WS2).** Deliverable: XGBoost and LightGBM models; Bi-LSTM comparison.
- **T17. Calibrate probabilities and define risk levels (WS2).** Deliverable: calibrated model, reliability plots, Green to Red thresholds.
- **T18. Implement persistence and hysteresis logic (WS2).** Deliverable: alert state machine tested on historical data.
- **T19. Train the RUL model (WS2).** Deliverable: RUL regressor with MAE and asymmetric score.
- **T20. Package the unified predict function and model registry (WS2).** Deliverable: versioned artifact, model card, MLflow runs.

Phase D: Explainability and Evaluation

- **T21. Implement SHAP explanations (WS3).** Deliverable: per-prediction top sensors and group-level contributions.

- **T22. Build the plain-language explanation generator (WS3).** Deliverable: explanation text with deviation direction and drift duration, labeled as hypotheses.
- **T23. Run event-level evaluation (WS3).** Deliverable: detection rate, lead time, and false-alarm table per held-out failure.
- **T24. Run ablation studies (WS3).** Deliverable: ablation table and discussion.
- **T25. Build the replay engine (WS3, WS7).** Deliverable: service that streams historical data at adjustable speed.

Phase E: Knowledge Base and RAG

- **T26. Collect and clean source documents (WS4).** Deliverable: document corpus with licensing and provenance notes.
- **T27. Author the FMEA matrix (WS4).** Deliverable: structured FMEA table reviewed against references.
- **T28. Chunk, embed, and index the corpus (WS4).** Deliverable: ChromaDB index with metadata.
- **T29. Implement hybrid retrieval and reranking (WS4).** Deliverable: retrieval module with measured hit rate.
- **T30. Implement alert-to-query construction (WS4).** Deliverable: function converting an alert into retrieval queries.
- **T31. Implement structured work-order generation (WS4).** Deliverable: validated JSON output with citations and safety rules.
- **T32. Render work-order PDFs (WS4).** Deliverable: PDF template and generator.
- **T33. Evaluate the RAG system (WS4).** Deliverable: gold set results for hit rate, faithfulness, and citation correctness.

Phase F: Backend and Data Layer

- **T34. Provision the database and apply migrations (WS7).** Deliverable: running TimescaleDB with hypertables and aggregates.
- **T35. Implement authentication and RBAC (WS6).** Deliverable: JWT login, httpOnly cookies, role checks.
- **T36. Implement the core REST endpoints (WS6).** Deliverable: pumps, telemetry, predictions, alerts, work orders.
- **T37. Integrate the inference service (WS6, WS2).** Deliverable: endpoint and worker producing predictions from telemetry.
- **T38. Implement the WebSocket stream (WS6).** Deliverable: live telemetry and prediction push per pump.

- **T39. Implement the alert pipeline (WS6).** Deliverable: alert creation, deduplication, RAG trigger, notification hook.
- **T40. Integrate the RAG service (WS6, WS4).** Deliverable: work-order endpoints backed by the RAG engine.
- **T41. Set up MQTT and the ingestion worker (WS7).** Deliverable: Mosquitto broker and a worker writing telemetry to the database.

Phase G: Frontend

- **T42. Replace mocks with real API calls and authentication (WS8).** Deliverable: login and live fleet overview.
- **T43. Build the pump detail page (WS8).** Deliverable: risk gauge, telemetry charts, anomaly timeline, RUL.
- **T44. Build the explanation view (WS8).** Deliverable: SHAP bar chart and sensor-group view.
- **T45. Build the alerts center and feedback flow (WS8).** Deliverable: filter, acknowledge, and feedback UI.
- **T46. Build the work-order screens (WS8).** Deliverable: view, edit, approve, and PDF download.
- **T47. Build the replay control panel (WS8).** Deliverable: start, pause, and speed controls wired to the replay API.
- **T48. Add Arabic and English support with RTL (WS8).** Deliverable: language switcher and translated interface.

Phase H: AI Assistant

- **T49. Implement the assistant tools against the backend (WS5).** Deliverable: tool functions matching the agreed schemas.
- **T50. Build the agent, system prompt, and guardrails (WS5).** Deliverable: agent with tool calling, refusal rules, and citation enforcement.
- **T51. Add session memory, streaming, and audit logging (WS5, WS6).** Deliverable: chat endpoint with streamed replies and logged tool calls.
- **T52. Build the assistant chat page (WS8).** Deliverable: chat UI with inline charts and citations.
- **T53. Evaluate the assistant (WS5).** Deliverable: test set of questions scoring tool-use correctness and absence of unsupported numbers.
- **T54. Add voice input and output (WS5, WS8).** Deliverable: speech-to-text and text-to-speech integration.

Phase I: IoT Prototype

- **T55. Assemble the hardware test bench (WS7).** Deliverable: pump loop with sensors wired and calibrated.
- **T56. Write the ESP32 firmware (WS7).** Deliverable: sampling and MQTT publishing with reconnection.
- **T57. Collect bench data and train the small edge model (WS2, WS7).** Deliverable: lightweight model on normal and fault-injected data.
- **T58. Connect the prototype to the platform (WS6, WS7).** Deliverable: live bench telemetry visible in the dashboard with alerts.

Phase J: Hardening and Delivery

- **T59. Run end-to-end integration tests (all).** Deliverable: automated test for telemetry, prediction, alert, work order, and assistant answer.
- **T60. Run load and failure tests (WS6, WS7).** Deliverable: results for streaming load, broker restart, and service failure recovery.
- **T61. Security review (WS6, WS7).** Deliverable: checklist covering auth, secrets, input validation, and transport security.
- **T62. Deploy to production (WS7, WS8).** Deliverable: Vercel frontend, hosted backend, managed database, documented environment variables.
- **T63. Set up monitoring and drift checks (WS7, WS3).** Deliverable: health dashboards and drift report.
- **T64. Optional: computer vision module (WS3 or WS5).** Deliverable: YOLOv8 model, endpoint, and dashboard page, only if the core is complete.
- **T65. Write the final report and user manual (all).** Deliverable: methodology, results with honest limitations, architecture, and user guide.
- **T66. Prepare the live demo (all).** Deliverable: scripted replay scenario showing early detection, explanation, work order, and an assistant conversation, plus the hardware demo and a recorded backup.
- **T67. Rehearse and finalize the presentation (all).** Deliverable: slides, poster, and a rehearsal with Q&A preparation.

14. Risks and Mitigations

- **Too few failure events for reliable evaluation:** use leave-one-failure-out, report per-event results, and avoid overstated claims.
- **Anonymous sensors limit physical interpretation:** present explanations as pattern-based hypotheses and validate against the FMEA.

- **Alert fatigue from false alarms:** calibration, persistence logic, and feedback-driven threshold tuning.
- **LLM hallucination in maintenance advice:** schema-constrained output, mandatory citations, safety text copied from sources, and human approval.
- **Integration delays between workstreams:** contracts frozen in Phase A, mock data for the frontend, and early integration milestones.
- **Scope creep (computer vision, voice, extra features):** core tasks T01 to T63 take priority; optional tasks only after the core passes integration tests.
- **Hardware delays or failures:** build the replay mode first so the demo never depends on hardware.

15. Deliverables Checklist

- Cleaned, versioned dataset and feature library.
- Trained and calibrated models with a model card and MLflow history.
- Event-level evaluation report with ablations.
- Explainability module and global SHAP analysis.
- Knowledge base, RAG engine, and work-order PDF generator.
- AI Assistant with tools, guardrails, and evaluation results.
- Backend API with authentication, WebSocket streaming, and OpenAPI documentation.
- Database schema and migrations.
- Frontend dashboard deployed on Vercel.
- IoT prototype and edge firmware.
- Dockerized deployment, CI pipeline, and monitoring.
- Final report, user manual, demo script, and presentation.